

Atividade de Recuperação

Verificação e Validação de Software

Sistema de Reserva de Salas de Estudo

Aluna: Lygian Monteiro
Disciplina: Verificação e Validação de Software
Data: 30/06/2025

1. Requisitos do Sistema

O sistema gerencia reservas de salas de estudo de uma instituição de ensino. Anteriormente as reservas eram feitas em planilhas pela secretaria, gerando problemas de duplicidade e falta de histórico. Os requisitos levantados foram:

1.1 Requisitos Funcionais (RF)

Código	Descrição
RF01	O sistema deve permitir o cadastro de um novo aluno.
RF02	O sistema deve permitir consultar as salas disponíveis.
RF03	O aluno deve poder realizar uma reserva de sala.
RF04	O sistema deve impedir reservas para horários já ocupados.
RF05	O sistema deve permitir consultar o histórico de reservas realizadas.

1.2 Requisitos Não Funcionais (RNF)

Código	Descrição
RNF01	O sistema deve ser implementado na linguagem Python.
RNF02	O código deve ser modularizado, com separação entre lógica e testes.
RNF03	O tempo de resposta das funções deve ser inferior a 3 segundos.

2. Plano de Testes Funcionais

Os casos de teste abaixo cobrem todos os requisitos funcionais. Os resultados obtidos são reais, gerados pela execução do pytest.

ID	Objetivo	Entrada	Resultado Esperado	Resultado Obtido	Status
CT01	Cadastrar aluno válido (RF01)	Matrícula='2024001', Nome='Ana Silva', senha='123456'	sucesso=True	sucesso=True	Atende
CT02	Rejeitar matrícula duplicada (RF02)	Matrícula='2024001' já cadastrada	sucesso=False, mensagem='Matrícula já cadastrada.'	sucesso=False, mensagem='Matrícula já cadastrada.'	Atende
CT03	Rejeitar cadastro com campos vazios (RF03)	Matrícula='', Nome='', senha=''	sucesso=False	sucesso=False para a maioria dos casos	Atende

ID	Objetivo	Entrada	Resultado Esperado	Resultado Obtido	Status
CT04	Consultar salas sem Data de Reserva (RF02)', Horário = 08:00 (as salas disponíveis retornou as 5 salas cadastradas	Data = 2025-07-01	Horário = 08:00	Retornou as 5 salas disponíveis	Aprovado
CT05	Realizar reserva válida (RF03), Aluno cadastrado, Sala disponível, Data e Horário válidos, mensagem de sucesso.	Aluno (RF03)	Sala disponível, Data e Horário válidos	Mensagem de sucesso.	Aprovado
CT06	Bloquear reserva duplicada (RF04), Sala 2, 2025-07-01, 10:00 e Sala 1, 2025-07-01, 08:00, mensagem de sucesso.	Sala 2, 2025-07-01, 10:00 e Sala 1, 2025-07-01, 08:00	Mensagem de sucesso.	Mensagem de sucesso.	Aprovado
CT07	Consultar histórico de aluno (RF05), Aluno cadastrado, 2 reservas realizadas	Aluno (RF05)	2 reservas realizadas	Retornou lista com 2 registros corretos	Aprovado
CT08	Histórico de aluno inexistente (RF05), Aluno não cadastrado	Aluno inexistente (RF05)	Aluno não cadastrado	Retornou []	Aprovado
CT09	Sala ocupada some (RF06), Sala disponível, Data e Horário válidos, mensagem de sucesso.	Sala disponível, Data e Horário válidos	Mensagem de sucesso.	Mensagem de sucesso.	Aprovado
CT10	Rejeitar reserva de Aluno não cadastrado (RF03), Sala disponível, Data e Horário válidos, mensagem de erro.	Aluno não cadastrado (RF03)	Sala disponível, Data e Horário válidos	Mensagem de erro.	Aprovado
CT11	Rejeitar reserva em Sala inexistente (RF03), Sala inexistente, Data e Horário válidos, mensagem de erro.	Sala inexistente (RF03)	Sala inexistente, Data e Horário válidos	Mensagem de erro.	Aprovado

3. Testes Automatizados (pytest)

Os testes foram escritos no arquivo **test_reserva_salas.py** utilizando o framework pytest. O sistema é implementado como a classe **SistemaReservas**; cada teste recebe uma instância nova e vazia através da **fixture sistema**, garantindo isolamento completo entre os casos (um teste nunca interfere no outro).

3.1 Trecho do código de testes

```
@pytest.fixture
def sistema():
    """Cria um sistema novo e vazio para cada teste (isolamento)."""
    return SistemaReservas()

# CT05 - RF03: Reserva válida é aceita
def test_reserva_valida(sistema):
    """Aluno cadastrado consegue reservar uma sala disponível."""
    sistema.cadastrar_aluno("2024001", "Ana Silva")
    resultado = sistema.realizar_reserva("2024001", "Sala 1", "2025-07-01", "08:00")
    assert resultado["sucesso"] is True
    assert "Sala 1" in resultado["mensagem"]

# CT06 - RF04: Reserva para horário já ocupado deve ser bloqueada
def test_reserva_horario_ocupado(sistema):
    """O sistema deve impedir uma segunda reserva no mesmo sala/data/horário."""
    sistema.cadastrar_aluno("2024001", "Ana Silva")
    sistema.cadastrar_aluno("2024002", "Bruno Costa")
    sistema.realizar_reserva("2024001", "Sala 2", "2025-07-01", "10:00")
    resultado = sistema.realizar_reserva("2024002", "Sala 2", "2025-07-01", "10:00")
    assert resultado["sucesso"] is False
    assert "já reservado" in resultado["mensagem"]
```

3.2 Resultado da execução do pytest

```
$ python -m pytest test_reserva_salas.py -v
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.1.1
collected 11 items

test_reserva_salas.py::test_cadastro_aluno_valido PASSED [ 9%]
test_reserva_salas.py::test_cadastro_matricula_duplicada PASSED [ 18%]
test_reserva_salas.py::test_cadastro_campos_vazios PASSED [ 27%]
test_reserva_salas.py::test_salas_disponiveis_sem_reservas PASSED [ 36%]
test_reserva_salas.py::test_reserva_valida PASSED [ 45%]
test_reserva_salas.py::test_reserva_horario_ocupado PASSED [ 54%]
test_reserva_salas.py::test_historico_reservas PASSED [ 63%]
test_reserva_salas.py::test_historico_aluno_inexistente PASSED [ 72%]
```

```
test_reserva_salas.py::test_sala_ocupada_nao_aparece_como_disponivel PASSED [ 81%]
test_reserva_salas.py::test_reserva_aluno_nao_cadastrado PASSED [ 90%]
test_reserva_salas.py::test_reserva_sala_inexistente PASSED [100%]

===== 11 passed in 0.06s =====
```

4. Cobertura de Testes

4.1 Relatório gerado pelo pytest-cov

```
$ python -m pytest test_reserva_salas.py --cov=reserva_salas --cov-report=term-missing

Name Stmts Miss Cover Missing
-----
reserva_salas.py 29 0 100%
-----
TOTAL 29 0 100%
```

4.2 Análise da cobertura

Cobertura atingida: 100% (todas as 29 linhas executáveis da lógica cobertas pelos testes).

O que está coberto: Todos os métodos da classe *SistemaReservas* foram exercitados — *cadastrar_aluno*, *consultar_salas_disponiveis*, *realizar_reserva* e *consultar_historico*. Foram testados tanto os caminhos de sucesso quanto todos os caminhos de erro (matrícula duplicada, campos vazios, conflito de horário, aluno não cadastrado e sala inexistente).

O que não está coberto: Apenas o bloco de demonstração `if __name__ == '__main__':` ficou de fora, o que é intencional — ele serve apenas para execução manual e foi marcado com `# pragma: no cover`, pois não faz parte da lógica de negócio testável.

5. Conclusão e Avaliação Crítica

O sistema desenvolvido atende a todos os cinco requisitos funcionais definidos no documento de requisitos. A implementação foi realizada em Python puro, sem dependências externas, respeitando o RNF01. A separação entre o módulo de lógica (*reserva_salas.py*, com a classe *SistemaReservas*) e o arquivo de testes (*test_reserva_salas.py*) atende ao RNF02. O tempo de execução dos 11 testes foi de 0,06 segundos, muito abaixo do limite de 3 segundos do RNF03.

A cobertura de 100% indica que todas as ramificações da lógica de negócio foram exercitadas pelos testes automatizados, incluindo os caminhos de sucesso e todos os caminhos de erro. Isso dá alta confiança de que o comportamento do sistema corresponde ao especificado nos requisitos.

Pontos de melhoria identificados: O sistema atualmente armazena dados em memória (listas e dicionários Python), o que significa que os dados são perdidos ao encerrar o programa. Uma evolução natural seria persistir os dados em banco de dados ou arquivo JSON. Além disso, não há validação de formato de data/horário, o que poderia causar reservas com entradas inválidas em um ambiente de produção.

Em síntese, o sistema cumpre seu papel pedagógico de demonstrar a aplicação de técnicas de Verificação e Validação: análise de requisitos, elaboração de plano de testes, automação com pytest e medição de cobertura.